

AI 桌面操控 Agent

认知-记忆混合架构技术方案

基于 EVOLVE 架构的进化式桌面自动化系统

课题方向：操作手册优化通用 Agent 操控桌面效率

汇报日期：2026 年 7 月 3 日

目 录

目 录	2
一、课题背景与问题分析	4
1.1 课题概述.....	4
1.2 核心问题.....	4
1.3 解决思路.....	4
二、EVOLVE 架构总体设计	4
2.1 架构设计思想.....	4
2.2 三层核心架构.....	5
2.3 Agent-Playbook 职责分离.....	5
三、核心模块详细设计	5
3.1 认知层（Cognitive Layer）	5
3.1.1 视觉感知模块.....	5
3.1.2 动作规划模块.....	6
3.1.3 决策路由.....	6
3.2 记忆层（Memory Layer）	6
3.2.1 Playbook 数据结构.....	6
3.2.2 语义匹配引擎.....	6
3.2.3 零 Token 回放机制.....	7
3.3 进化层（Evolution Layer）	7
3.3.1 Playbook 生成器.....	7
3.3.2 异常检测与修复.....	7
3.3.3 知识合并与去重.....	7

四、运行模式设计	8
五、Agent 集成方案	8
5.1 UI-TARS 集成	8
5.2 扩展接口.....	8
六、技术实现路线	9
七、设计演进	9
7.1 初始尝试.....	9
7.2 架构演进.....	9
7.3 当前版本特点.....	10

一、课题背景与问题分析

1.1 课题概述

本课题聚焦如何通过"操作手册"（Playbook）优化通用 Agent 操控电脑桌面的效率。随着大语言模型（LLM）和多模态模型的快速发展，AI Agent 在桌面自动化领域展现出巨大潜力，但在实际应用中仍面临诸多挑战。

1.2 核心问题

当前通用桌面操控 Agent 存在以下关键问题：

- (1) Token 消耗高：每次执行任务都需完整的视觉推理过程，API 调用成本高昂；
- (2) 执行不稳定：相同任务每次执行路径可能不同，缺乏确定性保障；
- (3) 经验无法积累：Agent 完成任务后，经验无法沉淀为可复用的操作知识；
- (4) 泛化与稳定性的矛盾：通用 Agent 追求泛化能力，但用户日常任务具有高度重复性。

1.3 解决思路

针对上述问题，本方案提出认知-记忆混合架构（EVOLVE），核心思路是：让通用 Agent 在执行任务的同时，将成功操作序列沉淀为结构化 Playbook；后续执行相同任务时，优先调用已验证的 Playbook 实现零成本回放，仅在遇到新场景时启用 Agent 的视觉推理能力。通过 Agent 与 Playbook 的职责分离与协同，实现效率与稳定性的统一。

二、EVOLVE 架构总体设计

2.1 架构设计思想

EVOLVE（Evolutionary Visual Operating Layer with Validated Execution）架构的设计思想源于人类学习过程的三阶段模型：认知（理解任务）、记忆（固化经验）、进化

（优化提升）。对应地，架构划分为认知层、记忆层和进化层三个核心层次，分别负责任务理解与推理、操作知识存储与回放、以及 Playbook 的持续优化。

2.2 三层核心架构

EVOLVE 架构由三个核心层次构成，各层职责明确、协同工作：

层次	英文名称	核心职责	关键技术
认知层	Cognitive Layer	任务理解、视觉感知、动作规划	UI-TARS 多模态 Agent、视觉语言模型
记忆层	Memory Layer	操作知识存储、快速检索、精确回放	结构化 Playbook、语义索引、参数模板
进化层	Evolution Layer	Playbook 优化、异常处理、知识合并	闭环学习、差异检测、版本管理

三层之间的关系为：认知层负责"第一次"如何完成任务；记忆层负责"第 N 次"如何高效复现；进化层负责"如何让第 N 次比第 1 次更好"。

2.3 Agent-Playbook 职责分离

架构的核心创新在于 Agent 与 Playbook 的职责分离设计。Agent 作为"老师"，仅在遇到新任务或 Playbook 失效时介入，负责视觉感知、推理决策和动作生成；Playbook 作为"学生笔记"，记录经过验证的操作序列，在常规场景下承担高效的执行引擎角色。两者通过进化层实现闭环：Agent 教一次，Playbook 记永远。

三、核心模块详细设计

3.1 认知层（Cognitive Layer）

认知层是系统的智能决策中枢，负责任务理解、环境感知和动作规划。当系统遇到新任务或已有 Playbook 无法覆盖的场景时，认知层启动工作。

3.1.1 视觉感知模块

视觉感知模块通过多模态大模型对桌面环境进行实时理解。系统截取当前屏幕图像，结合任务目标描述，由视觉语言模型生成对当前界面状态的语义描述，包括：可见元素识别、元素功能推断、当前页面状态判断。

3.1.2 动作规划模块

动作规划模块基于视觉感知结果和任务目标，生成具体的操作序列。每个动作包含：动作类型（点击、输入、滚动、等待等）、目标元素定位（坐标或选择器）、动作参数（输入文本、等待时长等）。动作序列经过执行验证后，将传递给记忆层进行结构化存储。

3.1.3 决策路由

认知层内置决策路由器，根据任务特征和历史执行情况，智能选择执行路径：优先检查记忆层是否存在匹配的 **Playbook**；若存在且上次执行成功，则直接调用记忆层回放；若不存在或上次执行失败，则启动 **Agent** 进行视觉推理。

3.2 记忆层（Memory Layer）

记忆层是系统的知识库，负责存储、检索和回放经过验证的操作序列。其核心理念是将 **Agent** 的成功经验固化为结构化、可参数化的 **Playbook**。

3.2.1 Playbook 数据结构

Playbook 采用分层结构设计，包含以下核心字段：

字段	类型	说明
task_signature	字符串	任务语义签名，用于匹配相似任务
steps	数组	操作步骤列表，每个步骤包含动作类型、目标、参数
selectors	对象	元素选择器集合，支持多策略定位（坐标/CSS/图像）
parameters	对象	可参数化变量（用户名、搜索关键词等）
success_count	整数	成功执行次数统计
last_executed	时间戳	上次执行时间
failures	数组	历史失败记录，用于进化层分析

3.2.2 语义匹配引擎

语义匹配引擎负责将用户任务描述与已有 **Playbook** 进行匹配。采用任务签名+语义相似度的双重匹配策略：精确匹配阶段对比任务签名字符串；模糊匹配阶段使用句

子嵌入模型计算语义相似度。匹配阈值根据任务类型动态调整，确保召回率与精确度的平衡。

3.2.3 零 Token 回放机制

当匹配到可用 Playbook 时，系统进入 Turbo 模式，直接按步骤顺序执行预设操作，全程不调用大模型 API，实现零 Token 消耗。执行过程中，系统通过 Playwright 或 PyAutoGUI 驱动实际操作，并通过截图对比验证每一步执行结果是否符合预期。

3.3 进化层（Evolution Layer）

进化层是系统的自我优化引擎，负责 Playbook 的持续改进和知识进化。其核心机制是闭环学习：每次 Agent 成功完成任务后，进化层自动将操作序列转化为标准 Playbook；每次 Playbook 执行遇到异常时，进化层记录失败信息并触发修复流程。

3.3.1 Playbook 生成器

Playbook 生成器将 Agent 的原始操作日志转换为标准化的 Playbook 文档。转换过程包括：去除冗余等待、抽象具体值为参数占位符、生成多策略元素选择器（主选择器+备用选择器）、添加验证点（检查执行后界面状态）。

3.3.2 异常检测与修复

当 Playbook 执行失败时，异常检测模块分析失败原因：界面变化导致的元素定位失败、操作流程变更导致的步骤不匹配、网络延迟导致的超时。针对不同原因，系统采取差异化修复策略：元素定位失败则启动 Agent 重新定位并更新选择器；流程变更则标记 Playbook 为待更新状态并通知进化层。

3.3.3 知识合并与去重

随着系统运行，可能产生多个相似任务的 Playbook。知识合并模块定期检测语义相似的 Playbook，评估是否可以合并为一个更通用的版本。合并过程保留最稳定的选择器策略，整合参数集合，并更新执行统计信息。

四、运行模式设计

系统支持三种运行模式，适应不同场景需求：

模式	触发条件	核心流程	特点
Turbo 模式	匹配到可用 Playbook 且上次成功	Playbook 直接回放，零 Token 消耗	最快、最便宜、最稳定
Checkpoint 模式	匹配到 Playbook 但需验证	Playbook 回放 + 关键节点截图验证	平衡效率与安全性
Agent 模式	无匹配 Playbook 或 Playbook 失效	Agent 完整视觉推理 + 动作规划	最智能、成本最高

三种模式之间可以自动切换：Turbo 模式执行失败时自动降级为 Checkpoint 模式；Checkpoint 验证不通过时降级为 Agent 模式；Agent 模式成功执行后，新 Playbook 进入记忆层，下次即可使用 Turbo 模式。

五、Agent 集成方案

系统设计采用开放的 Agent 集成架构，支持多种主流桌面 Agent 工具的接入。

5.1 UI-TARS 集成

UI-TARS 是字节跳动开源的多模态 Agent 模型，专为 UI 自动化设计。系统集成 UI-TARS 作为主要 Agent 引擎，通过 OpenAI 兼容 API 进行调用。配置方式支持本地 vLLM 部署和远程 API（如 WaveSpeedAI）两种模式，根据实际环境灵活选择。

5.2 扩展接口

除 UI-TARS 外，系统预留了标准化 Agent 接口，可接入其他 Agent 工具：

Agent 工具	状态	说明
UI-TARS	已集成	主要 Agent 引擎，多模态视觉推理
OpenCode	预留接口	计划集成的开源代码/桌面 Agent

Hermes	预留接口	计划集成的多模态 Agent 框架
--------	------	-------------------

Agent 桥接层负责统一不同 Agent 的输入输出格式，使上层业务逻辑与具体 Agent 实现解耦。新增 Agent 只需实现桥接接口即可无缝接入系统。

六、技术实现路线

计划分为四个阶段，逐步实现认知-记忆混合架构的完整功能：

阶段	周期	核心任务	交付物
第一阶段	第 1-2 周	基础框架搭建、Playwright/PyAutoGUI 集成、UI-TARS 接入	可运行的基础 Agent 框架
第二阶段	第 3-4 周	Playbook 数据结构设计与实现、语义匹配引擎开发	记忆层核心功能
第三阶段	第 5-6 周	进化层闭环学习、三种运行模式切换、异常处理	完整 EVOLVE 架构系统
第四阶段	第 7-8 周	端到端测试、场景验证、性能优化、文档完善	可演示的完整产品

技术栈选型：Python 3.10+作为开发语言，Playwright 处理浏览器自动化，PyAutoGUI 处理桌面自动化，SQLite/JSON 作为 Playbook 存储，Sentence-Transformers 提供语义向量能力，OpenAI 兼容 API 调用 UI-TARS。

七、设计演进

技术方案经历了从简单到复杂的演进过程，最终形成当前的认知-记忆混合架构。

7.1 初始尝试

开始尝试的是纯 LLM + Playbook 的基础架构。核心思路是：利用大语言模型（LLM）的指令理解和代码生成能力，将自然语言描述的任务直接转换为 Python 操作脚本，通过 Playwright 浏览器自动化引擎执行，并将成功执行的脚本以 Playbook 形式保存供后续复用。该架构包含三个核心组件：任务解析器（将用户指令转化为 LLM

Prompt)、代码生成器 (LLM 生成 Playwright 操作代码)、脚本仓库 (存储可复用的 Playbook 脚本)。

这一方案的动机源于对 LLM 代码能力的观察：LLM 不仅能理解自然语言，还能生成可直接运行的 Python 代码，这为实现桌面自动化提供了一条简洁路径。相比传统的 RPA 工具需要录制操作或编写复杂配置，该方案允许用户直接用自然语言描述任务，降低了使用门槛。同时，Playbook 的引入已经体现了经验复用的初步思想——将成功的操作脚本保存下来，避免每次都重新生成。

该方案具有以下优点：架构简单清晰，技术链路短，从任务描述到代码执行只需 LLM 生成和 Playwright 运行两个步骤；开发成本低，无需额外训练模型或构建复杂系统，仅需调用 LLM API 和集成 Playwright 即可运行；自然语言交互，用户无需学习特定语法或录制操作流程；可扩展性强，Playwright 本身支持丰富的浏览器操作，能够覆盖绝大多数 Web 场景。

但是有局限性：生成的 Playbook 是静态脚本，硬编码了元素选择器和操作参数，当目标网站界面发生任何变化（如按钮位置调整、CSS 类名变更）会失效；缺乏闭环机制，执行失败时无法自动诊断原因并修复等。

7.2 架构演进

后面重新思考了课题的核心目标：“通过“操作手册”（领域知识）优化通用 Agent 操控电脑桌面（浏览器/微信等客户端）的效率”，觉得方向有偏差，应该是通过操作手册让通用 Agent 更高效。基于这一认识，重新设计了当前的认知-记忆混合架构：

- (1) 引入 Agent-Playbook 职责分离：Agent 负责“教”，Playbook 负责“记”和“做”；
- (2) 设计三层架构：认知层处理新任务，记忆层高效复现，进化层持续优化；
- (3) 实现三种运行模式：Turbo 零 Token 回放、Checkpoint 验证回放、Agent 智能规划；
- (4) 建立闭环学习机制：Agent 教一次，Playbook 记永远，实现经验的持续积累。

7.3 当前版本特点

相比初始方案，当前版本具有以下优势：稳定性增强（经过验证的 Playbook 确保确定性执行）；自我进化能力（Agent 经验自动沉淀为可复用知识）；开放架构设计

(支持多种 Agent 工具的灵活接入)。后续工作将聚焦于端到端测试和实际场景验证，持续完善系统功能。

八、部分输出和问题

首次执行：python main.py --stage zhihu --task "登录知乎"

```
connector: <aihttp.connector.TCPConnector object at 0x000002C725353910>
PS E:\fagogo\project> python main.py --stage zhihu --task "写AI文章配图发布" --title "AI" --topic "人工智能"
=====
知乎阶段 - EVOLVE架构
=====
2026-07-02 20:34:22.126 INFO | core.metrics:start_task:123 - Metrics tracking started: task='写AI文章配图发布' stage='zhihu'
2026-07-02 20:34:22.127 INFO | core.llm:_init_:97 - LLMClient initialized: model=deepseek-chat, base_url=https://api.deepseek.com/v1
2026-07-02 20:34:22.134 INFO | core.playbook:_load_all_into_cache:309 - Loaded 7 playbooks into cache
2026-07-02 20:34:22.134 INFO | core.playbook:_init_:102 - PlaybookManager initialized: storage_dir=playbooks
2026-07-02 20:34:22.134 INFO | core.executor:setup:190 - Starting browser: headless=False, slow_mo=100ms
2026-07-02 20:34:22.135 INFO | core.executor:setup:196 - Loading storage_state from: ./playbooks/zhihu/session.json
2026-07-02 20:34:24.673 INFO | core.executor:setup:217 - Browser started successfully
2026-07-02 20:34:24.674 INFO | core.task_analyzer:_init_:142 - TaskAnalyzer initialized
2026-07-02 20:34:24.726 DEBUG | core.llm:chat:184 - LLM request: model=deepseek-chat, messages=1, temperature=0.7
2026-07-02 20:34:31.131 INFO | core.llm:chat:222 - LLM response received: prompt_tokens=113, completion_tokens=395, cost=0.0009 CNY
2026-07-02 20:34:31.132 DEBUG | core.metrics:record_llm_usage:161 - LLM usage recorded: input=113, output=395, cost=0.0009 CNY
2026-07-02 20:34:31.140 INFO | core.playbook:query:134 - Playbook query: task='知乎写文章-AI...' stage='zhihu' -> 3 results
2026-07-02 20:34:31.140 INFO | core.dispatcher:_decide_mode:164 - Playbook success rate 90% -> Mode A (Turbo)
[*] 调度决策: turbo 模式
    Playbook: 命中
    成功率: 90%
    版本: v1

[Mode A: Turbo Replay - 零感知高速回放]
[1/5] navigate: 写文章页面
2026-07-02 20:34:36.160 INFO | core.executor:execute:279 - Executed [navigate] on '写文章页面': Navigated to https://zhuanlan.zhihu.com/write
[验证] url_contains: 'zhuanlan.zhihu.com/write...'
2026-07-02 20:34:36.161 DEBUG | core.verifier:verify:121 - 验证 #1: url_contains = 'zhuanlan.zhihu.com/write'
[/] 验证通过
[2/5] wait: 编辑器加载
2026-07-02 20:34:36.277 INFO | core.executor:execute:279 - Executed [wait] on '编辑器加载': Waited for: textarea[placeholder*='标题']
[3/5] input: 标题输入框
2026-07-02 20:34:36.626 INFO | core.executor:execute:279 - Executed [input] on '标题输入框': Input 'AI' into textarea[placeholder*='标题']
[4/5] click: 正文编辑器
2026-07-02 20:34:36.843 INFO | core.executor:execute:279 - Executed [click] on '正文编辑器': Clicked: [contenteditable='true']
[5/5] input: 正文内容
2026-07-02 20:34:37.107 INFO | core.executor:execute:279 - Executed [input] on '正文内容': Input '2023年，ChatGPT让全世界都体验了一把“魔法时刻”。但一年过去，比起惊叹AI能写诗、编程、做PPT，更值得追问的是：**当机器越来越像人，人该如何自处？**'

## 技术狂飙背后的“认知陷阱”

很多人把AI当作“更快的搜索引擎”或“高级自动补全”，这其实低估了它的颠覆性。大语言模型真正可怕之处，在于它模糊了“知识”与“幻觉”的边界。它能用最自信的语气说出最离谱的答案——比如建议你“用胶水固定披萨上的芝士”。这意味着，**AI时代最稀缺的能力，不是提问，而是“辨别”**。我们不再需要记住所有答案，但必须学会拆解逻辑、验证事实、追问“为什么”。

## 人的“不可替代性”反而更清晰了

当AI能写80分的文案、画70分的图，人类的价值反而从“产出”转向了“判断”。你不需要比AI写得快，但要能指出哪里不对；你不需要比AI画得美，但要能定义“美”的方向。**真正的护城河，是那些AI学不会的东西：基于肉身经验的直觉、对复杂人性的共情、在模糊两难中拍板的勇气。就像医生看CT片子，AI能标出99%的病灶，但最后需要人类决定：这个病
```

```
Windows PowerShell
很多人把AI当作“更快的搜索引擎”或“高级自动补全”，这其实低估了它的颠覆性。大语言模型真正可怕之处，在于它模糊了“知识”与“幻觉”的边界。它能用最自信的语气说出最离谱的答案——比如建议你“用胶水固定披萨上的芝士”。这意味着，**AI时代最稀缺的能力，不是提问，而是“辨别”**。我们不再需要记住所有答案，但必须学会拆解逻辑、验证事实、追问“为什么”。

## 人的“不可替代性”反而更清晰了

当AI能写80分的文案、画70分的图，人类的价值反而从“产出”转向了“判断”。你不需要比AI写得快，但要能指出哪里不对；你不需要比AI画得美，但要能定义“美”的方向。**真正的护城河，是那些AI学不会的东西：** 基于肉身经验的直觉、对复杂人性的共情、在模糊两难中拍板的勇气。就像医生看CT片子，AI能标出99%的病灶，但最后需要人类决定：这个病人，要不要冒风险动手术？

## 总结

AI不是对手，而是一面镜子。它照出我们过去引以为傲的“知识搬运”其实多么脆弱，也逼我们重新审视：**什么是只有“人”才能承担的责任？** 答案或许很简单：选择与承担。算法可以给出最优解，但只有人，能为这个选择付出代价、赋予意义。' into [contentaditable='true']
2026-07-02 20:34:42.060 | DEBUG | core.playbook:update_stats:235 - Updated stats for pb_zhihu_write_article: count=2, success_rate=100.00%
2026-07-02 20:34:42.061 | DEBUG | core.metrics:record_step:141 - Step recorded: duration=4.86s, success=True

=====
🔥 执行结果
=====
模式: turbo
状态: ✓ 成功
步数: 5
LLM调用: 0
耗时: 4.9秒
消息: Turbo完成: 5/5 步成功
=====
2026-07-02 20:34:42.064 | INFO | core.metrics:finish_task:196 - Task finished: success=True, total_duration=19.94s, steps=2, retries=0

📊 总Metrics:
耗时: 19.9s
步骤: 2/2
Token: 508
费用: ¥0.0009
2026-07-02 20:34:42.066 | INFO | core.executor:teardown:221 - Closing browser...
2026-07-02 20:34:42.739 | INFO | core.executor:teardown:235 - Browser closed
Unclosed client session
client_session: <aihttp.client.ClientSession object at 0x0000021106A68D0>
Unclosed connector
connections: ['deque([(<aihttp.client_proto.ResponseHandler object at 0x0000021106AEC7D0>, 16493.843))']
connector: <aihttp.connector.TCPConnector object at 0x0000021106A68D0>
PS E:\fagogo\project>
```

第 2 次执行：tokens 消耗为 0

```
PS E:\fagogo\project> python main.py --stage zhihu --task "登录知乎"
知乎阶段 - EVOLVE架构
=====
2026-07-02 21:38:36.085 | INFO | core.metrics:start_task:123 - Metrics tracking started: task='登录知乎' stages='zhihu'
2026-07-02 21:38:36.086 | INFO | core.llm:__init__:97 - LLMClient initialized: model=deepseek-chat, base_url=https://api.deepseek.com/v1
2026-07-02 21:38:36.109 | INFO | core.playbook:load_all_into_cache:309 - Loaded 5 playbooks into cache
2026-07-02 21:38:36.109 | INFO | core.playbook:__init__:102 - PlaybookManager initialized: storage_dir=playbooks
2026-07-02 21:38:36.109 | INFO | core.executor:setup:190 - Starting browser: headless=False, slow_mo=100ms
2026-07-02 21:38:36.109 | INFO | core.executor:setup:196 - Loading storage_state from: ./playbooks/zhihu/session.json
2026-07-02 21:38:37.927 | INFO | core.executor:setup:219 - Browser started successfully
2026-07-02 21:38:37.928 | INFO | core.task_analyzer:__init__:142 - TaskAnalyzer initialized
[*] Task: 知乎登录
[*] Session file path: E:\fagogo\project\playbooks\zhihu\session.json
[Step 0/4] 检查当前登录状态...
[*] Found session file: E:\fagogo\project\playbooks\zhihu\session.json
2026-07-02 21:38:41.159 | INFO | core.executor:execute:279 - Executed [navigate] on 'Navigate to https://www.zhihu.com': Navigated to https://www.zhihu.com
2026-07-02 21:38:41.160 | DEBUG | core.metrics:record_step:141 - Step recorded: duration=3.07s, success=True
[*] Session restored from file, already logged in
2026-07-02 21:38:43.004 | DEBUG | core.playbook:update_stats:235 - Updated stats for pb_zhihu_login: count=2, success_rate=100.00%
2026-07-02 21:38:43.004 | INFO | core.playbook:record:171 - Updated existing playbook: pb_zhihu_login
[*] Playbook recorded: 知乎登录 (1 steps)
[*] 登录成功 (或已自动恢复登录)
2026-07-02 21:38:43.005 | INFO | core.metrics:finish_task:196 - Task finished: success=True, total_duration=7.32s, steps=1, retries=0

📊 总Metrics:
耗时: 7.3s
步骤: 1/1
Token: 0
费用: ¥0.0000
2026-07-02 21:38:43.007 | INFO | core.executor:teardown:221 - Closing browser...
2026-07-02 21:38:43.007 | INFO | core.executor:teardown:235 - Browser closed
```

python main.py --stage zhihu --task "写 AI 文章配图发布，搜索并评论点赞收藏" --title "AI 发展现状" --topic "人工智能的发展历程"


```
2026-07-02 20:34:42.060 | DEBUG | core.playbook:update_stats:235 - Updated stats for pb_zhihu_write_article: count=2, success_rate=100.00%
2026-07-02 20:34:42.061 | DEBUG | core.metrics:record_step:141 - Step recorded: duration=4.86s, success=True

=====
⚡ 执行结果
=====
模式: turbo
状态: ✓ 成功
步数: 5
LLM调用: 0
耗时: 4.9秒
消息: Turbo完成: 5/5 步成功
=====
2026-07-02 20:34:42.064 | INFO | core.metrics:finish_task:196 - Task finished: success=True, total_duration=19.94s, steps=2, retries=0

📊 总Metrics:
耗时: 19.9s
步骤: 2/2
Token: 508
费用: ¥0.0009
2026-07-02 20:34:42.066 | INFO | core.executor:teardown:221 - Closing browser...
2026-07-02 20:34:42.739 | INFO | core.executor:teardown:235 - Browser closed
Unclosed client session
client_session: <aiohttp.client.ClientSession object at 0x0000021106A686D0>
Unclosed connector
connections: ['deque([(<aiohttp.client_proto.ResponseHandler object at 0x0000021106AEC7D0>, 16493.843))']]
connector: <aiohttp.connector.TCPConnector object at 0x0000021106A688D0>
```

在发布文章时会页面出现下面问题知乎报错，无法发表。

AI 桌面操控 Agent 认知-记忆混合架构技术方案

